



# Algoritmos II

*Prof. Dr. Eliseu LS*  
*(11) 9 4034-0057 WhatsApp*

# **Apresentação Pessoal**

***Prof. D.r Eliseu Lemes da Silva***

## ***FORMAÇÃO ACADÊMICA ATUAL:***

- 1. Pós-doutorado em Educação ênfase em Psicologia Cognitiva (UFLO/AR)***
- 2. Doutorado em Educação ênfase em Inclusão Social (UF/MG)***
- 3. Doutorado em Humanidades e Artes Ênfase em Informática/Educação Superior***
- 4. Especialização em Engenharia da Computação (UFU/MG)***
- 5. Licenciatura em Matemática (UMESP)***
- 6. Graduação em Processamento de Dados (UNESP)***
- 7. Bacharel em Direito (IME/SP)***

## ***ATIVIDADES PROFISSIONAIS ATUAIS:***

- 1. Coordenador de Curso Superior de Tecnologia em ADS (CEETEPS/FATEC-SP)***
- 2. Professor Titular na área de Tecnologia da Informação (CEETEPS/FATEC-SP)***
- 3. Professor Titular na área de Tecnologia da Informação (SENAC/SP)***
- 4. Pesquisador / Escritor de literatura sobre Educação.***

**Curriculo Lattes: <http://lattes.cnpq.br/9300104925024179>**

# EMENTA DO CURSO

Aborda o desenvolvimento de algoritmos, a partir de técnicas de programação interativa e recursiva e do estudo de eficiência de algoritmos. Apresenta os métodos de busca e classificação de dados em memória. Aborda, ainda, o conceito e implementação de vetores e matrizes, estruturas de dados simples: fila, pilha, fila de prioridade e heap.

## **Bibliografia básica:**

**CORMEN, T. H. et al. Algoritmos: teoria e prática. Rio de Janeiro: Campus, 2002.**

**FEOFILOFF, P. Algoritmos em linguagem C. Rio de Janeiro: Elsevier, 2009.**

**SZWARCFITER, J. L; MARKENZON, L. Estruturas de dados e seus algoritmos. Rio de Janeiro: LTC, 2010.**

## **OUTRAS FONTES:**

- Notas de aula Dr. Eliseu LS: Slides, exercícios de aprendizagem e vídeo-aulas.
- Sites referência: <https://www.devmedia.com.br/guia/linguagem-java/38169>

○

[www.oracle.com](http://www.oracle.com)

## ANÁLISE SOBRE O APRENDIZADO



# METODOLOGIA

## Entrega dos Tarefas / ADOs;

1. Recursos obrigatórios: Caderno, Computador e Internet;
2. As entregas ou códigos dos programas serão entregues pela ferramenta adotada na disciplina ( **Blackboard** ) sempre dentro de um arquivo no formato MS-WORD contendo - Nome dos autores, enunciado, código e prints de execução ou de diagramas usados;
3. Cada atividade irá receber uma pontuação de 2 até 4 pontos com prazo de entrega de até uma semana, salvo exceções, tarefas atrasadas só valem (50%) 2 pontos e não serão aceitas após 3 dias de atraso;
4. **Ao final do semestre** será atribuída uma nota entre 0 (Zero) e 10 (Dez) conforme percentual de pontos obtidos nas ADOs e Laboratórios que será usada para composição da média final junto com as demais avaliações;

## Dinâmica da Aula:

1. ***Aula com slides, modelos, tutoriais e exercícios de aprendizagem:*** Nesta parte, o professor apresenta o princípio de cada conceito dos algoritmos ou comandos em lousa e/ou projeção em forma de simulação ou de apresentação, por vezes simulando os algoritmos passo a passo de modo que os alunos possam acompanhar tranquilamente.
2. ***Laboratórios e ADOs :*** Nesta parte da aula, caberá aos alunos a prática dos exercícios em laboratório com a supervisão do professor, por vezes em forma Laboratórios (Exercícios individuais) e/ou ADOs (Geralmente tarefas realizadas em grupo).
3. ***Forma de correção de ADOs / Laboratórios - Exercícios de Fixação:*** O processo de correção das atividades se dará de forma individualizada e pessoal durante o processo de construção da atividade onde a pontuação irá aparecer após a entrega pela plataforma com prazo de entrega previamente fixado.

# Correção de ADOs / Laboratórios

## **a) A nota do código das avaliações semanais perderá pontos quando :**

1. o enunciado não for atendido na íntegra;
2. o código não código não contiver pacotes, bibliotecas ou a diretivas necessárias;
3. o código não não funcionar;
4. Número de sub rotinas ou métodos estiver diferente do que foi pedido no enunciado;

## **a) O código dos métodos sub rotinas será considerado totalmente errado quando:**

1. A lista de parâmetros ou argumentos não estiver de acordo com o enunciado;
2. Os parâmetros e argumentos estiverem incorretos ou declarados sem necessidade;
3. contiver fórmulas erradas ou montadas de forma incorreta

# ENTREGA DE ADOs e Laboratórios

Basta criar um arquivo do tipo DOCX ou PDF, colocar o enunciado de cada exercício, o código fonte e a imagem do print da tela de execução de cada código fonte que você compilou e executou.

Se for um Diagrama, coloque apenas o enunciado e logo após a imagem de cada diagrama.

## Exemplo de Entrega da ADO 1

1. Abra um arquivo em branco no formato DOCX e salve como ADO1.docx
2. Se for uma lista de programas, coloque o enunciado de cada programa, seguido do código fonte correto e por último um print da tela de execução, faça isso para todos os programas, porém coloque todos dentro de um único arquivo em DOCX ou PDF;
3. Se for uma lista de diagramas ou imagens, coloque o enunciado seguido do diagrama ou imagem tudo dentro de um único arquivo no formato DOCx ou PDF;
4. Para entregar converta para PDF ou envie no mesmo formato DOCX pelo Blackboard.
5. ADO feita em grupo deve ser entregue apenas uma cópia para o grupo inteiro, não é necessário a entrega individual;

# DATAS / COMPOSIÇÃO DA MÉDIA

## Avaliações AV1, AV2, SUB e ADOs:

**AV1:** ( 7 aula ) Avaliação individual, escrita e sem consulta, possui peso  $\frac{1}{3}$  da média, onde será avaliado o conhecimento Teórico e/ou prático do aluno;

**AV2:** ( 16 aula ) Avaliação individual, escrita e sem consulta, possui peso  $\frac{1}{3}$  da média, onde será avaliado os conhecimentos acumulados teóricos e/ou práticos dos alunos;

**ADOs:** Exercícios individuais (Laboratórios valendo um 1 ponto) e coletivos (Ados em grupo de 3 alunos valendo 4 pontos). Laboratórios entrega obrigatória no mesmo dia e Ados prazo de 7 dias, permitindo atrasos de apenas 3 dias com decremento de 50% da pontuação, ou seja, valendo só dois pontos para trabalhos atrasados.

PESOS DAS NOTAS : AV1 = 30%, AV2 = 30% e ADOs = 40%.

$$\text{MÉDIA} = \text{AV1} * 0.3 + \text{AV2} * 0.3 + \text{ADOs} * 0.3$$

Para a aprovação o aluno deverá obter no mínimo a média 6.





**(Java)**  
**Algoritmos II**  
**Aula 1**

*Prof. Dr. Eliseu LS*



# ***( Java )*** ***Vetores em Java***

*Prof. Dr. Eliseu LS*

# Arquivos Necessários

- JVM = apenas a virtual machine, esse download não existe, ela sempre vem acompanhada.
- JRE = **Java Runtime Environment**, ambiente de execução Java, formado pela JVM e bibliotecas, tudo que você precisa para executar uma aplicação Java. Mas nós precisamos de mais.
- JDK = **Java Development Kit**: Nós, desenvolvedores, faremos o download do JDK do Java SE (Standard Edition). Ele é formado pela JRE somado a ferramentas, como o compilador.

Tanto o JRE e o JDK podem ser baixados do site <http://www.oracle.com/technetwork/java/>. Para encontrá-los, acesse o link Java SE dentro dos top downloads. Consulte o apêndice de instalação do JDK para maiores detalhes.

**JAVA SE Download :** <https://www.oracle.com/java/technologies/downloads/#java20>

"Atenção verifique o sistema operacional da sua máquina e também se o processador é de 32 ou 64 bits. "

# Escolha do IDE de trabalho

IDE em português significa - Ambiente Integrado de Desenvolvimento, é um software que permite a edição e compilação de códigos fontes em uma determinada linguagem de programação, abaixo estão alguns IDEs sugeridos para Java.

- **Netbeans ( Nosso IDE escolhido )**
- Eclipse (Na internet)
- Jcreator (versão gratuita ou paga)
- Visual Studio Code
- Soluções on-line: repl.it ou openGDB

**APACHE NETBEANS Download:** <https://netbeans.apache.org/download/index.html>

"Atenção verifique o sistema operacional da sua máquina e também se o processador é de 32 ou 64 bits. "

# Componentes de um Código Java

## Package/Pacote

Conjunto de classes dentro de um namespace, trata-se de um arquivo compactado que contém diversas classes similares ou com funcionalidades parecidas é como se fosse uma pasta que contém classes com funcionalidades similares.

## Import

Semelhante ao comando *#include da linguagem C*. Através do comando *import*, pode-se utilizar outras classes e seus métodos, dentro da classe que está sendo construída.

## Objeto

Um elemento ou entidade que possui características e funções, por exemplo, um celular tem suas características, como cor, peso, marca, etc e têm também funções, como ligar, receber chamadas, escutar músicas, visualizar vídeos e documentos, etc.

## Classe

É um objeto modelado para programação orientada a objetos, uma classe possui **ATRIBUTOS** que são as variáveis e **MÉTODOS** que são as funções de uma classe.

# Conhecimentos necessários para esta aula

1. Conceito de Array()/Vetores()
2. Laços while() e for()
3. Importação de Pacotes java
4. Conversão de String para Números

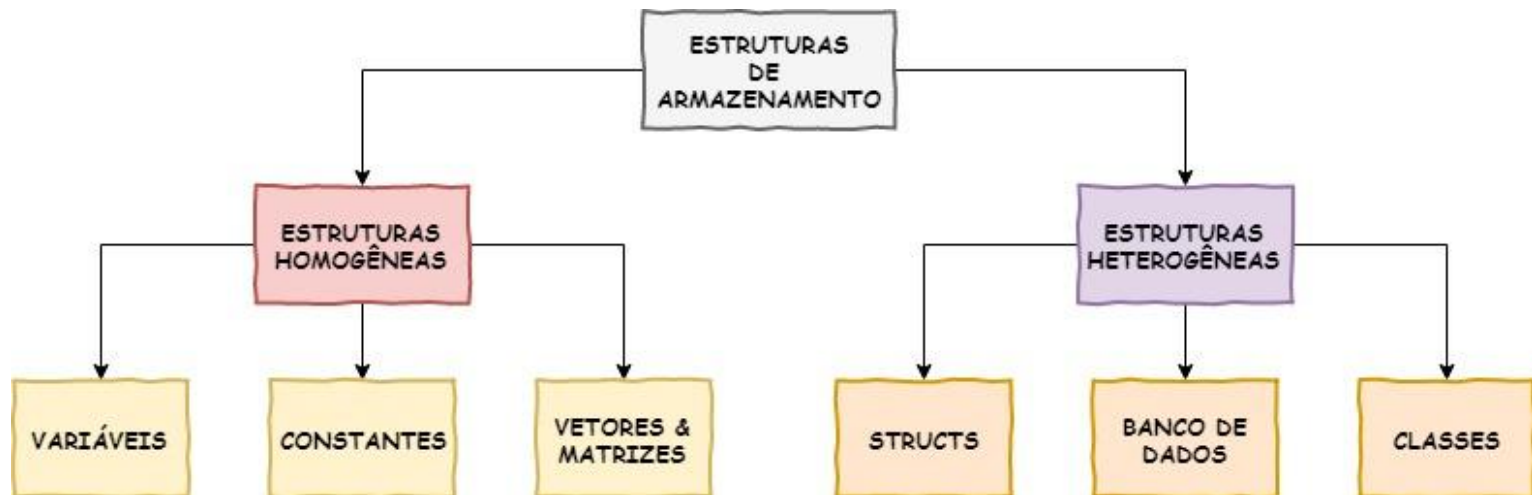
# Conteúdo desta aula de Vetores

1. Uso da classe JOptionPane e o pacote javax.swing;
2. Definição, leitura, cálculo e exibição envolvendo vetores em Java;
3. Métodos showMessageDialog() e showInputDialog();

# Estruturas de Armazenamento

Estruturas **homogêneas** permitem o armazenamento de apenas um datatype por vez, como é o caso das matrizes, vetores, variáveis e constantes. Estruturas **heterogêneas** permitem o armazenamento de vários datatypes em uma única estrutura, como é o caso dos registros (structs), das tabelas em banco de dados, classes, etc.

Figura 1 - Estruturas de Armazenamento



Font: SILVA, Eliseu Lemes, 2023.



# Vetor (ARRAY)

Conhecido por nós como **Array**, um **Vetor** é um uma lista homogênea de dados, isto é, representa um conjunto de dados onde cada elemento possui um índice que vai de 0 (zero) até (n-1) onde n será a quantidade total de elementos do conjunto.

## [ Criando Vetores Preenchidos ]

Estes vetores já são criados com dados, o tamanho irá depender da quantidade de elementos do vetor.

Por Exemplo:

```
int v [ ] = { 1, 0 , -1 };
```

n = total de elementos = 3

1 está na posição 0

0 está na posição 1

-1 está na posição (n - 1) = 2

| Vetor v [ ] |       |
|-------------|-------|
| índice      | dados |
| 0           | 1     |
| 1           | 0     |
| 2           | -1    |

## [Criando Vetores não Preenchidos]

Define-se o nome e tamanho, os dados são inseridos depois.

```
int [] x;   int x[];  
x = new int [ 3 ]; // três números  
x [0] = 5; x [1] = 10;
```

```
string [ ] nome; ou String nome[ ];
```

```
nome = new String [3];
```

```
int i = 1; nome [ i ] = "Maria";  
i = 2; nome [ i ] = "João";
```

```
nome [ 0 ] = "Sara";
```

# Composição de Strings

A função format da classe String permite a formatação e composição de strings em java, para cada datatype existe um caractere de composição precedido pelo caractere %. Vejamos o exemplo abaixo:

```
public class Aula1 {  
    public static void main(String[] args) {  
        String nom = "Sara", sobr= "Silva";  
        double sal = 9450.99;  
        String saida;  
        saida=String.format("A Sra %s %s recebe R$ %.2f", nom,sobr, sal);  
        System.out.println(saida);  
    }  
}
```

# Caracteres de Composição Java (%)

| Format Specifier | Data Type                                                   | Output                                                                                                                   |
|------------------|-------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------|
| %a               | floating point (except <i>BigDecimal</i> )                  | Returns Hex output of floating point number.                                                                             |
| %b               | Any type                                                    | "true" if non-null, "false" if null                                                                                      |
| %c               | character                                                   | Unicode character                                                                                                        |
| %d               | integer (incl. byte, short, int, long, bigint)              | Decimal Integer                                                                                                          |
| %e               | floating point                                              | decimal number in scientific notation                                                                                    |
| %f               | floating point                                              | decimal number                                                                                                           |
| %g               | floating point                                              | decimal number, possibly in scientific notation depending on the precision and value.                                    |
| %h               | any type                                                    | Hex String of value from hashCode() method.                                                                              |
| %n               | none                                                        | Platform-specific line separator.                                                                                        |
| %o               | integer (incl. byte, short, int, long, bigint)              | Octal number                                                                                                             |
| %s               | any type                                                    | String value                                                                                                             |
| %t               | Date/Time (incl. long, Calendar, Date and TemporalAccessor) | %t is the prefix for Date/Time conversions. More formatting flags are needed after this. See Date/Time conversion below. |
| %x               | integer (incl. byte, short, int, long, bigint)              | Hex string.                                                                                                              |

# Simulação com Valores Preenchidos - Classe1.java

O algoritmo a seguir examina uma lista para separar o maior e o menor valor dentro de um vetor previamente preenchido. ( Favor colocar o void main() para executar )

## // Classe1.java

```
public class Classe1 {
    public static int[] xy = {1000, 5000,4900, 7000};
    public static String nome [] = {"João", "Pedro", "Sara",
"leia"};
    public static void mostrar() {        int imaior = 0, imenor = 0;
        int tot = xy.length;// pega o tamanho
        for (int i = 0; i < tot; i++) {
            if (i == 0) {    imaior = i;    imenor = i;}
            if (xy[i] > xy[imaior]) {    imaior = i; }
            if (xy[i] < xy[imenor]) {    imenor = i;  }}

    System.out.println (String.format("valor maior %d  %s", xy[imaior],
nome[imaior] ));
    System.out.println (String.format("valor menor %d  %s", xy[imenor],
nome[imenor] ));}
    public static void main(String[] args) { mostrar(); } }
```

# Simulação com leitura de vetores Classe2.java

O algoritmo a seguir trabalha com leitura de vetores do tipo string e double. Para leitura e exibição usamos a classe JOptionPane do pacote javax.swing; Calcula a média de uma lista de alunos através da leitura de notas.

## // Classe2.java

```
import javax.swing.JOptionPane;

public class Classe2 {    public static int linha = -1;
    public static String nome[] = new String[20];
    public static double nota1[] = new double[20];
    public static double nota2[] = new double[20];
    public static double media[] = new double[20];

    public static void ler(String nom, Double n1, Double n2) {
        if (linha >= 20) {    return; }
        linha++;
        nome[linha] = nom;    nota1[linha] = n1;    nota2[linha] = n2;
        media[linha] = (n1 + n2) / 2;    }

    public static void exhibir() {        String saida = "";

        for (int i = 0; i <= linha; i++) {    saida += nome[i] + "    " + media[i] + "\n";
        }

        JOptionPane.showMessageDialog (null, saida);        }

    public static void main(String[] args) {
        String nome = JOptionPane.showInputDialog ("Digite nome:");
        double n1 = Double.parseDouble (JOptionPane.showInputDialog (null, "Nota 1:"));
        double n2 = Double.parseDouble (JOptionPane.showInputDialog (null, "Nota
2non:"));
        ler(nome, n1, n2);        exhibir();    }}
```

## ***Laboratório 1***

- a) Incremente a Classe2, faça um menu de controle para inserir vários nomes e várias notas
- b) Usando JOptionPane para leitura e exibição, faça os exercícios A, B, C, D e E da página 74 do livro de exercícios usando os conceitos ensinados na aula 1 que incluem, composição de strings, declaração e uso de vetores, etc.

**NOTA:** Entrega obrigatória hoje pelo Blackboard valendo 1,0 ponto em um arquivo no formato DOCX.